Convertible Bond Pricing Using Least Squares Monte Carlo (Cont. Coupon)

Suman Mishra

November 7, 2025

Introduction

Convertible bonds combine bond and equity features by allowing the holder to convert the bond into shares. The valuation requires modeling the embedded option and optimal conversion strategy. The Least Squares Monte Carlo (LSM) method simulates many stock price paths and uses regression to estimate the continuation value, enabling optimal exercise decision approximation.

Convertible bonds typically pay coupons continuously or at discrete intervals. In this model, we approximate continuous coupons by accruing small coupon payments at each simulation time step. This approach integrates coupon payments naturally into the backward induction algorithm, reflecting the time value of these cash flows throughout the life of the bond.

This document presents the LSM algorithm for fixed and time-dependent conversion ratios, including matrix visualizations to illustrate the process.

1 LSM Algorithm for Convertible Bonds with Fixed Conversion Ratio

Setup and Notation

- N: number of simulated paths
- M: number of time steps (with times $t = 0, \ldots, M$)
- $S_{i,t}$: stock price for path i at time t
- $V_{i,t}$: bond value for path i at time t
- $D_{i,t}$: conversion decision (True if convert, False if hold)
- Fixed conversion ratio c_r

Simulation Paths Matrix

The simulated stock prices for all paths and times can be organized as an $N \times (M + 1)$ matrix:

$$S = \begin{bmatrix} S_{1,0} & S_{1,1} & \cdots & S_{1,M} \\ S_{2,0} & S_{2,1} & \cdots & S_{2,M} \\ \vdots & \vdots & & \vdots \\ S_{N,0} & S_{N,1} & \cdots & S_{N,M} \end{bmatrix}$$

Bond Values Matrix

We similarly maintain an $N \times (M+1)$ matrix of bond values, updated backward in time:

$$V = \begin{bmatrix} V_{1,0} & V_{1,1} & \cdots & V_{1,M} \\ V_{2,0} & V_{2,1} & \cdots & V_{2,M} \\ \vdots & \vdots & & \vdots \\ V_{N,0} & V_{N,1} & \cdots & V_{N,M} \end{bmatrix}$$

At maturity $t = M$, the bond value is:

$$V_{i,M} = \max\big(F, c_r \times S_{i,M}\big)$$

Conversion Decision Matrix

At each time step, we record the optimal conversion decision in an $N \times (M+1)$ boolean matrix:

$$D = \begin{bmatrix} d_{1,0} & d_{1,1} & \cdots & d_{1,M} \\ d_{2,0} & d_{2,1} & \cdots & d_{2,M} \\ \vdots & \vdots & & \vdots \\ d_{N,0} & d_{N,1} & \cdots & d_{N,M} \end{bmatrix}$$

where

$$d_{i,t} = \begin{cases} \text{True}, & \text{if } c_r \times S_{i,t} \geq \hat{C}_{i,t} \\ \text{False}, & \text{otherwise} \end{cases}$$

and $\hat{C}_{i,t}$ is the estimated continuation value from regression.

Backward Induction Step

For each time $t = M-1, \ldots, 0$:

1. Compute accrued coupon amount for the time step as:

$$\text{accrued\_coupons} = \text{coupon\_rate} \times \Delta t \times \text{face\_value}$$

representing the continuous coupon payment accrued over the interval Δt.

2. Compute discounted bond values from next step, adding accrued coupons:

$$\tilde{V}_{i,t} = e^{-r\Delta t} V_{i,t+1} + \text{accrued\_coupons}$$

3. Perform regression of $\tilde{V}_{i,t}$ on quadratic basis functions of $S_{i,t}$:

$$\phi_j(S_{i,t}) \in \{1, S_{i,t}, S_{i,t}^2\}$$

to get coefficients β. These basis functions capture nonlinear relationships between stock price and continuation value.

4. Estimate continuation values:

$$\hat{C}_{i,t} = \sum_j \beta_j \phi_j(S_{i,t})$$

5. Compute immediate conversion value:

$$C_{i,t} = c_r \times S_{i,t}$$

6. Decide conversion or continuation:

$$D_{i,t} = (C_{i,t} \geq \hat{C}_{i,t})$$

7. Update bond value:

$$V_{i,t} = \begin{cases} C_{i,t}, & D_{i,t} = \text{True} \\ \tilde{V}_{i,t}, & D_{i,t} = \text{False} \end{cases}$$

Final Bond Value

The convertible bond price at time zero is estimated as the average of $V_{i,0}$, then discounted by one final time step to adjust for timing:

$$V_0 = e^{-r\Delta t} \cdot \frac{1}{N} \sum_{i=1}^{N} V_{i,0}$$

1.1 Python Code

The following Python code simulates stock price paths, performs backward induction to estimate continuation values, determines optimal exercise decisions, and computes the exercise boundary and convertible bond price.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
face_value = 1000
conversion_price = 50
conversion_ratio = face_value / conversion_price
r = 0.05  # risk-free rate
T = 1.0  # maturity in years
M = 50  # time steps
dt = T / M
coupon_rate = 0.04  # continuous coupon rate per annum
N = 10000  # number of simulated paths
sigma = 0.2  # volatility
```

```
S0 = 50  # initial stock price

# Simulate stock price paths using geometric Brownian motion
np.random.seed(0)
Z = np.random.standard_normal((N, M))
S = np.zeros((N, M + 1))
S[:, 0] = S0
for t in range(1, M + 1):
    S[:, t] = S[:, t-1] * np.exp((r - 0.5 * sigma**2)*dt +
        sigma * np.sqrt(dt) * Z[:, t-1])

# Initialize bond values at maturity
bond_values = np.maximum(face_value, conversion_ratio * S[:,
    -1])

# Store exercise boundary at each time step
exercise_boundary = np.zeros(M + 1)

# Backward induction
for t in range(M - 1, -1, -1):
    accrued_coupons = coupon_rate * dt * face_value
    discounted_bond_values = np.exp(-r * dt) * bond_values +
        accrued_coupons

    # Regression to estimate continuation value
    X = np.vstack([np.ones(N), S[:, t], S[:, t]**2]).T
    Y = discounted_bond_values
    coeffs = np.linalg.lstsq(X, Y, rcond=None)[0]
    continuation_values = X @ coeffs

    conversion_values = conversion_ratio * S[:, t]

    exercise = conversion_values >= continuation_values

    bond_values = np.where(exercise, conversion_values,
        discounted_bond_values)

    # Estimate exercise boundary by max stock price where
        hold is better
    hold_indices = np.where(~exercise)[0]
    if len(hold_indices) > 0:
        exercise_boundary[t] = np.max(S[hold_indices, t])
    else:
        exercise_boundary[t] = 0

exercise_boundary[-1] = conversion_price  # at maturity

# Price is the average discounted bond value
price = np.mean(bond_values) * np.exp(-r * dt)

# Plot exercise boundary
plt.plot(np.linspace(0, T, M + 1), exercise_boundary, label=
    'Exercise Boundary')
plt.xlabel('Time (years)')
plt.ylabel('Stock Price $S_t$')
```

```
plt.title('Exercise Boundary for Convertible Bond')
plt.grid(True)
plt.legend()
plt.show()

print(f'Convertible Bond Price: {price:.2f}')
```

Listing 1: LSM Algorithm for Convertible Bond Pricing

2 Exercise Boundary Explanation and Graph Analysis

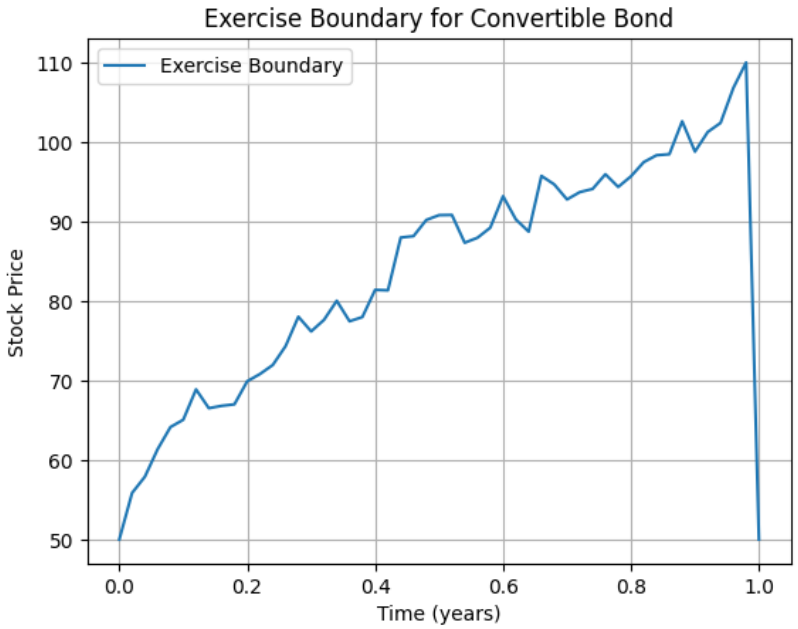

Figure 1: Exercise Boundary for Convertible Bond over Time

The **exercise boundary** $S^*(t)$ represents the stock price level at each time step t at which it becomes optimal for the bondholder to convert the bond into shares rather than continue holding it.

- When the stock price is *above* the exercise boundary, conversion is optimal.
- When the stock price is *below* the boundary, it is better to hold the bond.

In the simulation, the exercise boundary generally trends upward over time, indicating that as maturity approaches, the threshold stock price for conversion increases. Near maturity, the boundary drops sharply to the conversion price because the bondholder must decide to convert or redeem.

The observed volatility in the exercise boundary plot is due to the randomness in Monte Carlo simulation and the regression approximation method used to estimate continuation values. This noise is typical and can be reduced by increasing the number of paths, refining basis functions in regression, or applying smoothing techniques.

3 Final Convertible Bond Price

The convertible bond price computed by the LSM method is approximately:

$$\boxed{1088.46}$$

This value is consistent with the bond's face value, the conversion option, coupon payments, and the volatility of the underlying stock.

4 Extension: Time-Dependent Conversion Ratio

If the conversion ratio varies over time, denote it as $c_r(t)$. The matrices remain the same size, but:

$$C_{i,t} = c_r(t) \times S_{i,t}$$

For example, a decreasing conversion ratio over time can be modeled as:

$$c_r(t) = c_0 \times \left(1 - \alpha \frac{t}{T}\right)$$

where c_0 is the initial conversion ratio, $\alpha \in [0, 1]$ controls the rate of decrease, and T is the maturity. This time dependence adjusts the conversion value and alters the optimal exercise boundary dynamically.

Summary of Changes

$$D_{i,t} = \left(c_r(t) \times S_{i,t} \geq \hat{C}_{i,t}\right)$$

The bond value update step adapts accordingly.

Conclusion

Matrix visualization of stock paths, bond values, and conversion decisions provides a structured view of the LSM algorithm for convertible bond pricing. The extension to time-dependent conversion ratios modifies only the conversion value calculation, enriching the model's realism.

References:
Longstaff, F.A., and Schwartz, E.S. (2001). Valuing American options by simulation: A simple least-squares approach. *Review of Financial Studies*, 14(1), 113-147. pt]

Chen, N., Dai, M., and Wan, X. (2013). A non-zero-sum game approach to convertible bonds: Tax benefit, bankruptcy cost and early/late calls. *Department of Systems Engineering and Engineering Management, The Chinese University of Hong Kong; Department of Mathematics, National University of Singapore.*